

コマ 5: if/else + 比較演算

今日のゴール

条件分岐と比較演算を実装する。

第 04 回までは、関数内の文は常に先頭から順に実行された。この回では、条件によって実行する文を変える `if / else` を扱う。

```
int main() {  
    int a;  
    int b;  
    a = 10;  
    b = 3;  
    if (a > b) {  
        return a - b;  
    } else {  
        return b - a;  
    }  
}
```

目標は、条件式の結果に応じて、`then` 側か `else` 側のどちらかの文だけを実行できるようにすることである。

1 この回で扱う範囲

対象にするプログラムは、`main` 関数内に以下の要素を含むものに限定する。

種類	例
比較演算	<code>a > b, x == 42, y <= 10</code>
if 文	<code>if (cond) { ... }</code>
if/else 文	<code>if (cond) { ... } else { ... }</code>
else if	<code>if (cond1) { ... } else if (cond2) { ... }</code>
入れ子の if	<code>if (cond1) { if (cond2) { ... } }</code>

種類	例
ブロック	<code>{ ... }</code>
return 文	<code>return 式;</code>
変数宣言・代入・算術式	第 04 回までに扱ったもの

while、for、関数呼び出し、ポインタ、配列はまだ扱わない。

2 AST を確認する

まず、if/else を含むプログラムがどのような AST になるか確認する。

```
python3 scaffold/parse_viewer.py sessions/05_if_else/tests/target.c
```

このプログラムの内容は次の通り。

```
int main() {
    int a;
    int b;
    a = 10;
    b = 3;
    if (a > b) {
        return a - b;
    } else {
        return b - a;
    }
}
```

実行すると、次のような S 式が表示される。

```
(program
 (funcdef "main" :type int (params)
  (block (decl "a" :type int) (decl "b" :type int)
   (exprstmt
    (assign (var "a") (num 10)))
   (exprstmt
    (assign (var "b") (num 3)))
```

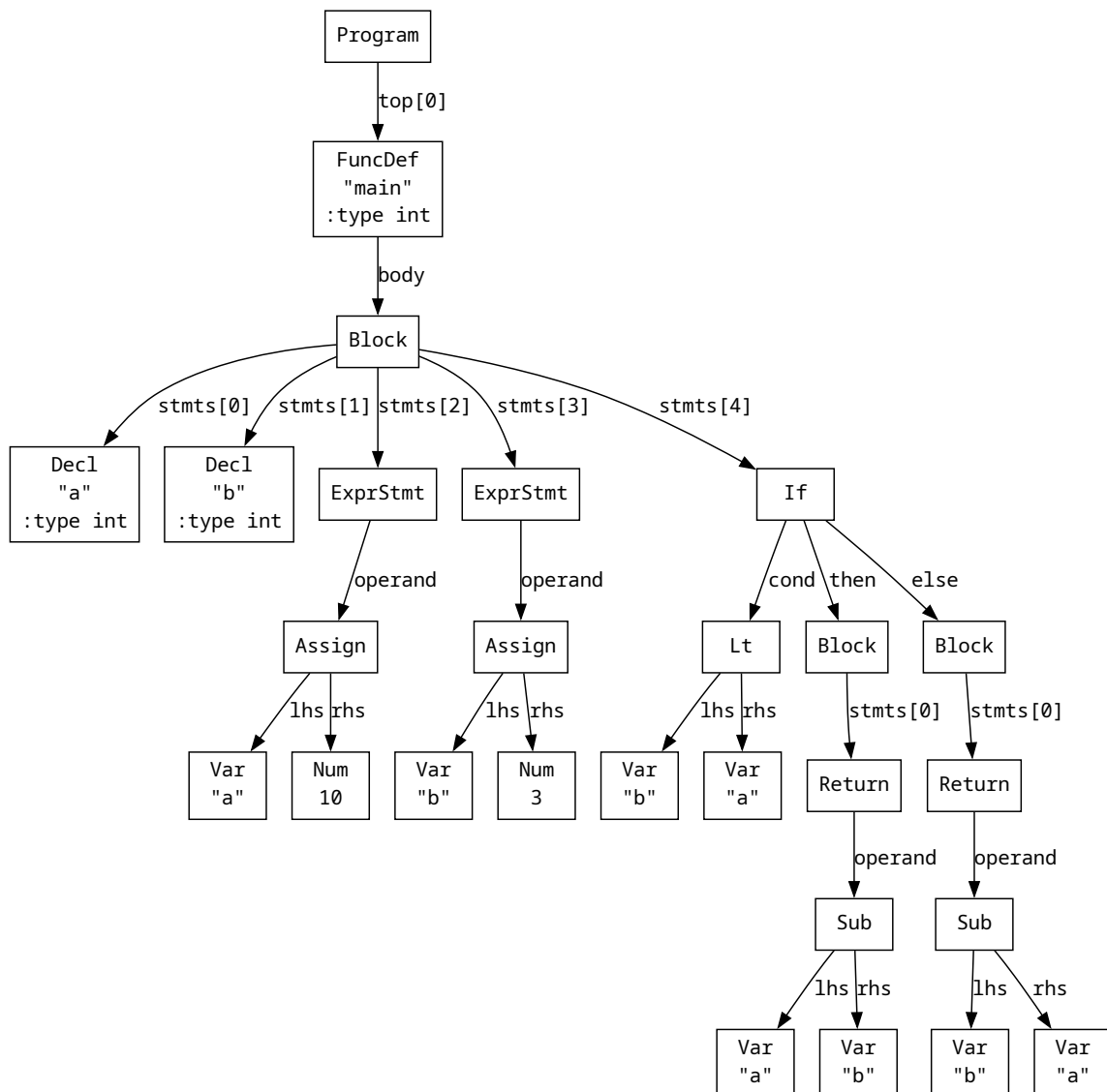


図 1 if/else を含むプログラムの AST

```

(if
  (cond
    (lt (var "b") (var "a")))
  (then
    (block
      (return
        (sub (var "a") (var "b")))))
  (else
    (block
      (return
        (sub (var "b") (var "a"))))))))

```

注目すべき点は 2 つある。

1 つ目。元の C コードの条件式は `a > b` だが、AST では `(lt (var "b") (var "a"))` として表されている。Parser は `>` を `<` に、`>=` を `<=` に、それぞれ左右を入れ替えて変換する。そのため、AST 上では `ND_GT` や `ND_GE` は決して現れない。

2 つ目。if ノードは次の 3 つのフィールドを持つ。

フィールド	役割
<code>cond</code>	条件式ノード
<code>then</code>	条件が真のときに実行する文ノード
<code>else_</code>	条件が偽のときに実行する文ノード。 <code>else</code> がない場合は <code>None</code>

3 比較演算

この回では、比較演算も式として扱う。比較演算の結果は、真なら `1`、偽なら `0` とする。

Parser が返す比較演算ノードは次の通り。

C の式	AST	意味
<code>a == b</code>	<code>ND_EQ</code>	等しいなら <code>1</code>
<code>a != b</code>	<code>ND_NE</code>	等しくなければ <code>1</code>
<code>a < b</code>	<code>ND_LT</code>	左辺が右辺より小さければ <code>1</code>
<code>a <= b</code>	<code>ND_LE</code>	左辺が右辺以下なら <code>1</code>
<code>a > b</code>	<code>ND_LT</code> に正規化	<code>b < a</code> として表される
<code>a >= b</code>	<code>ND_LE</code> に正規化	<code>b <= a</code> として表される

`ND_GT` や `ND_GE` は登場しないため、これらに対応するコードは不要である。

4 比較演算のコード生成

二項演算の共通パターンにより、比較命令を出す直前には次の状態になっている。

レジスタ	内容
a1	左辺の値
a0	右辺の値

比較演算は次のように生成する。

ノード	生成する命令	説明
ND_EQ	<code>sub a0, a1, a0 / seqz a0, a0</code>	差が 0 なら <code>a0 = 1</code> 、それ以外は 0
ND_NE	<code>sub a0, a1, a0 / snez a0, a0</code>	差が 0 でなければ <code>a0 = 1</code> 、差が 0 なら 0
ND_LT	<code>slt a0, a1, a0</code>	<code>lhs < rhs</code> なら <code>a0 = 1</code> 、それ以外は 0
ND_LE	<code>slt a0, a0, a1 / xori a0, a0, 1</code>	<code>!(rhs < lhs)</code> として計算する

`seqz` (set if equal to zero)、`snez` (set if not equal to zero)、`slt` (set if less than)、`xori` (xor immediate) は RV64 の命令である。

たとえば、`a < b` の AST は `(lt (var "a") (var "b"))` であり、次のように生成される。

```
# a1 = a, a0 = b の状態
slt a0, a1, a0
```

`a1 < a0` が成り立てば `a0 = 0`、成り立てば `a0 = 1` となる。

5 条件分岐の考え方

C では、`if` の条件式の値が `0` なら偽、`0` 以外なら真として扱う。

```
if (cond) {  
    文  
}
```

このコードは、次のような流れに変換できる。

```
cond を計算する  
もし cond の結果が 0 なら Lend に飛ぶ  
文を実行する  
Lend:
```

RV64 では、`beqz a0, label` を使う。

```
beqz a0, .L1
```

これは、`a0` が `0` なら `.L1` にジャンプする、という意味である。`cond` の計算結果は `a0` に入っているため、`beqz a0, Lend` で「偽なら飛ばす」という動作になる。

6 ラベル生成

分岐の飛び先には、一意なアセンブリラベルが必要になる。

`new_label()` は、呼ばれるたびに異なる名前のラベルを返すメソッドである。

```
def _new_label(self) -> str:  
    self._label_counter += 1  
    return f".L{self._label_counter}"
```

インスタンス変数 `self._label_counter` は `__init__` で `0` 初期化される。

`.L` で始まるラベルはローカルラベルと呼ばれ、アセンブラが自動的に一意性を保証する。

7 if の生成パターン

```
if (cond) {  
    文  
}
```

これは、次のような命令の並びとして生成する。

```
# cond を codegen  
beqz a0, Lend  
# 文  
Lend:
```

条件が偽なら `beqz` で `Lend` に飛び、文は実行されない。

8 if/else の生成パターン

```
if (cond) {  
    then 文  
} else {  
    else 文  
}
```

この場合は、`then` 側の実行後に `else` 側を飛ばす必要がある。

```
# cond を codegen  
beqz a0, Lelse  
# then  
j Lend  
Lelse:  
# else  
Lend:
```

こうしないと、`then` 側の実行後に `else` 側にも突入してしまう。

偽のときは `beqz` でラベルへ飛ぶ。if/else では、`then` 側の最後に `j Lend` を置いて `else` 側を飛び越す。

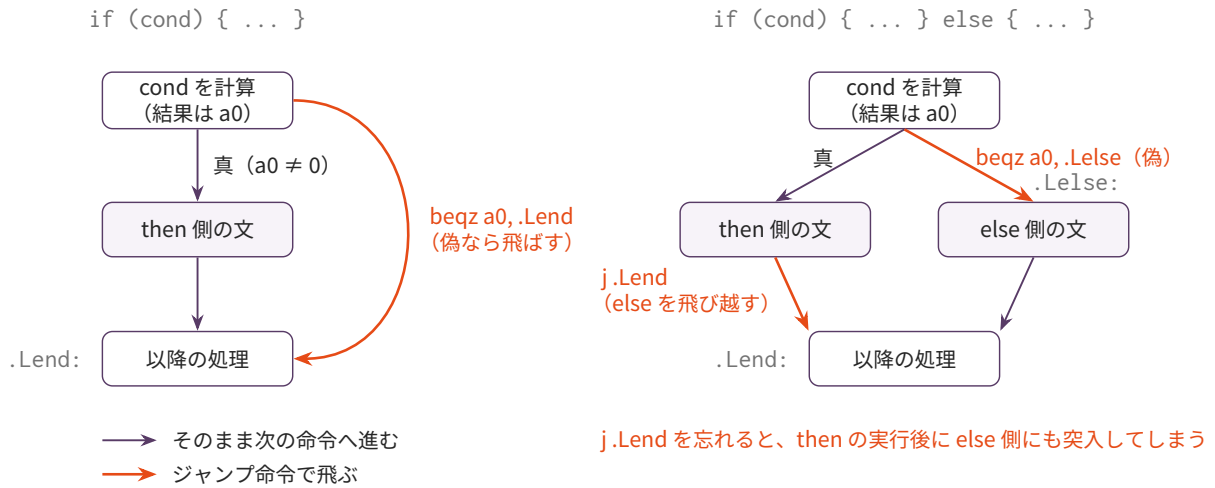


図2 if と if/else の制御フロー

9 else if の扱い

`else if` という専用ノードはない。Parser は、`else` の中にさらに `if` が入っているものとして扱う。

たとえば、次のコードを考える。

```
if (score >= 90) {
    return 4;
} else if (score >= 70) {
    return 3;
} else {
    return 1;
}
```

AST では、`else` の中にもう一つの `if` が入る。

```
(if
 (cond ...)
 (then ...)
 (else
  (if
   (cond ...)
   (then ...)
   (else ...))))
```


`else_` が指すノードの `kind` が `ND_IF` であるかどうかにかかわらず、`gen_stmt(stmt.else_)` を再帰的に呼べば、`else if` も自然に処理できる。`else_` が `None` になるまで、再帰が続く。

10 return と共通エピローグ

第 04 回までは、`return` 文が関数の末尾にしか現れなかったため、`return` の後に直接エピローグを置けばよかった。

しかし、第 05 回からは `if` の中などに `return` が現れる。

```
if (a > b) {  
    return a;  
}  
return b;
```

この場合、`if` の中の `return` で関数をその場で終了しなければならない。そのため、`return` 文では、戻り値を `a0` に入れた後、関数末尾のエピローグラベルへジャンプする。

```
return 式を codegen する  
j Lreturn
```

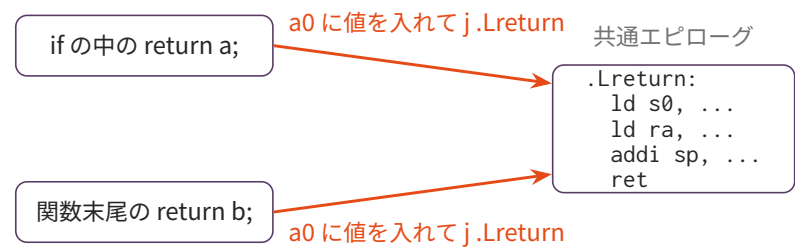
`gen_func()` では、関数ごとに `_ret_label` を作り、エピローグの直前に置く。

```
.Lreturn:  
    ld s0, ...  
    ld ra, ...  
    addi sp, sp, ...  
    ret
```

共通エピローグラベルを使うことで、関数内のどこに `return` があっても正しく関数を抜けられるようになる。

11 gen_stmt に追加する処理

コマ 4 で実装した `gen_stmt()` に、さらに文の種類を追加する。



関数内のどこで return しても、必ず共通のエピローグを通して関数を抜ける

図 3 複数の `return` が共通エピローグに合流する

node.kind	呼ばれる handler	処理
'Block'	gen_stmt_Block	stmts を先頭から順に self.gen_stmt() する
'If'	gen_stmt_If	条件式を self.codegen() し、ラベルを生成して分岐。 else_ があれば self.gen_stmt() で処理する
'Break'	gen_stmt_Break	コマ 6 で対応。 コマ 5 では未対応でよい
'Continue'	gen_stmt_Continue	同上

12 編集するファイル

- mycc.py

`importlib` でコマ 4 の `Codegen04` を継承した `Codegen05` に、以下の機能を追加する。

実装対象	役割
<code>_new_label()</code>	一意なアセンブリラベルを返す
<code>gen_stmt_If(node)</code>	条件分岐 (<code>beqz</code> / <code>j</code>) を生成。 <code>else_</code> も処理する
<code>gen_stmt_Block(node)</code>	<code>stmts</code> を順に <code>gen_stmt</code> する
<code>codegen_Eq(node)</code> / <code>codegen_Ne(node)</code> / <code>codegen_Lt(node)</code> / <code>codegen_Le(node)</code> (dispatcher 対応)	比較演算。handler を追加する <code>codegen</code> に <code>'Eq'</code> <code>'Ne'</code> <code>'Lt'</code> <code>'Le'</code> の case を追加する

13 tests/

ファイル	内容	期待値
<code>target.c</code>	<code>if (a > b)</code> の <code>else</code> 分岐	7
<code>if_only.c</code>	<code>if</code> のみ (<code>else</code> なし)	1
<code>compare.c</code>	<code>==</code> , <code>>=</code> , <code><</code> の組み合わせ	42
<code>nested.c</code>	入れ子 <code>if</code>	8

`if_true.c` | `if` の条件が真になる場合 | 対応する `.ans` を参照 |

`if_false.c` | `if` の条件が偽になる場合 | 対応する `.ans` を参照 |

`if_elseif.c` | `else if` の連なり | 対応する `.ans` を参照 |

`nested_if.c` | `if` の入れ子 | 対応する `.ans` を参照 |

14 テスト

```
python3 scaffold/test_runner.py sessions/05_if_else
```

tests/target.c がコンパイルでき、終了コード 7 になれば基本形は成功。

個別に動かす場合は、次のようにする。

```
python3 sessions/05_if_else/mycc.py sessions/05_if_else/tests/target.c \
| riscv64-linux-gnu-gcc -x assembler -static - -o out

qemu-riscv64 ./out
echo $?
```

15 注意

Parser は `>` と `>=` を正規化するため、`ND_GT` や `ND_GE` は AST 上に現れない。これらのノードに対応するコードは不要である。

`ND_IF` の `cond` は、比較演算に限らず、任意の式を取れる。C では、`0` が偽、`0` 以外が真である。

補足

&& と || の注意: このスキAFFォールドの `&&` / `||` は、本物の C と違い両辺を必ず評価する（短絡評価をしない）。そのため、`if (p != 0 && p->val > 0)` のように「左辺が偽なら右辺を評価しないこと」に頼った書き方はできない（`p` が `NULL` でも `p->val` を読んでしまう）。条件は 2 つの `if` に分けて書くこと。短絡評価の実装は発展課題 S1 で扱う。